Московский Физико–Технический Институт

Численные методы
Задание №10

ФИО: Лазарь Владислав Игоревич
Группа: Б01-207а
Почта: lazar.v@phystech.edu

Долгопрудный, 2026 г.

Оптимизация генераторов псевдослучайных чисел

В данной работе рассматривается задача высокопроизводительной генерации псевдослучайных чисел для методов Монте–Карло. Традиционные скалярные генераторы, такие как `minstd_rand`, становятся неэффективны при необходимости генерации сотен миллионов значений, так как накладные расходы на вызов функции и последовательная зависимость состояний ограничивают пропускную способность.

Цель работы — реализация векторного варианта линейного конгруэнтного генератора (`LCG`) с поддержкой SIMD-инструкций и параллельных вычислений через механизм `skip-ahead`.

Линейный конгруэнтный метод

Базовым алгоритмом выбран генератор `std::minstd_rand`, описываемый рекуррентным соотношением:

$$x_{i+1} = (A \cdot x_i) \mod M,$$

где параметры стандарта `minstd`:

- $A = 48271$ (множитель),
- $M = 2^{31} - 1 = 2147483647$ (модуль, простое число Мерсенна).

Для получения равномерно распределенной величины u_i применяется нормирующее отображение $[0;\ M) \to [0;\ 1)$:

$$u_i = \frac{x_i}{M}.$$

Проблема производительности. Векторизация

Основная проблема скалярной реализации — операция взятия остатка от деления (`%`), которая имеет высокую латентность (до десятков тактов процессора), и невозможность параллельного вычисления следующего состояния x_{i+1} без знания текущего x_i.

Для векторизации используется свойство линейности `LCG`. Можно перепрыгнуть через k шагов за одну операцию, используя предварительно вычисленное значение A_k:

$$x_{i+k} = (A_k \cdot x_i) \mod M, \quad \text{где } A_k = A^k \mod m.$$

Если хранить состояние генератора не как одно число, а как вектор $\boldsymbol{X} = [x_i, x_{i+1}, \ldots, x_{i+k-1}]$, то следующий блок состояний $\boldsymbol{X}'$ может быть вычислен поэлементно:

$$\boldsymbol{X}' = (A_k \cdot \boldsymbol{X}) \mod m.$$

Поскольку операции над элементами вектора независимы, компилятор может заменить цикл на SIMD-инструкции, обрабатывая 8–16 чисел за один такт.

Быстрое вычисление модуля

Так как $M = 2^{31} - 1$, то дорогостоящее деление заменяется битовыми операциями. Для любого $y < M^2$ справедливо:

$$y \mod m = (y \gg 31) + \big(y \,\&\, (2^{31} - 1)\big).$$

Если результат превышает M, из него единожды вычитается M. В коде это реализовано функцией `fast_mod`:

```
static uint32_t fast_mod(uint64_t y) {
    uint32_t z = static_cast<uint32_t>((y >> 31) + (y & M_HEX));
    return (z >= M) ? (z - M) : z;
}
```

Параллелизм. `Skip-Ahead`

Для распараллеливания вычислений на T потоках необходимо гарантировать, что каждый поток генерирует уникальную, непересекающуюся подпоследовательность. Используется метод `Skip-Ahead`.

Пусть общий объем выборки N. Каждый поток t $(0 \leqslant t < T)$ должен сгенерировать $\frac{N}{T}$ точек. Поскольку на одну точку требуется 2 случайных числа, смещение по состоянию для потока t составляет:

$$\text{offset}_t = t \cdot \frac{2N}{T}.$$

Начальное состояние для потока t вычисляется как:

$$x_{\text{start}}^{(t)} = (A^{\text{offset}_t} \cdot x_0) \mod m.$$

Это позволяет инициализировать все потоки независимо от одного `seed`, избегая гонок данных и использования `mutex`.

Реализация

Разработан класс `VecLCG_FP32`, включающий:

1. **Векторное состояние**: массив `state_` размера K (степень двойки, например, 256).
2. **Предвычисление констант**: A_K для шага вектора и A_{offset} для параллельного старта.
3. **Генерация блоками**: метод `fill_float` сначала копирует текущие значения в выходной буфер, затем обновляет все состояния векторным умножением на A_K.
4. **Локальность данных**: размер блока подобран так, чтобы массив состояний и буфер результатов помещались в `L1`-кэш.

Для проверки корректности реализован побитовый тест, сравнивающий вывод векторного генератора со скалярным эталоном.

Методология измерений

Язык и компилятор: `C++17`, компилятор `g++` со следующими флагами оптимизаций:

```
-O3 -fopenmp -ffast-math -fopenmp
```

.

Эксперименты запускались на платформе с архитектурой `AMD Zen 3`:

- Процессор: `AMD Ryzen 7 5800U` (8 ядер, 16 потоков)
- Микроархитектура: `Zen 3 (Cezanne)`.
- Операционная система: `Linux Ubuntu`.
- Тактовая частота: базовая 1.9 ГГц. Фиксировалась с помощью:

```
cpupower frequency-set -min 1.9GHz -max 1.9GHz
```

Результаты экспериментов

Эксперименты проводились на системе с многоядерным процессором. Объем выборки для расчета π: $N = 10^8$ точек ($2 \cdot 10^8$ случайных чисел).

1. Функциональная корректность

Проведена проверка первых 10^6 значений. Векторная реализация полностью совпала со скалярным `std::minstd_rand`.

```
> ./verify_bitwise
Bitwise correctness verified for 1000000 values
```

2. Производительность и точность Monte Carlo

Запуск бенчмарка с использованием 16 потоков `OpenMP` и размером вектора 256.

```
> ./monte_carlo_vec
Estimated pi : 3.1415510400
Reference pi : 3.1415926536

Abs error    : 0.0000416136
Std error    : 0.0001642183

Time         : 0.0651630970 s
Cycles/rand  : 0.6180628750
```

Абсолютная ошибка ($4.16 \cdot 10^{-5}$) значительно меньше стандартного отклонения и находится в интервале 3σ.

Показатель производительности 0.618 `CPE` свидетельствует о практически полном использовании векторных мощностей процессора. Время выполнения 0.065 с для 100 млн точек.